

Representación de la información

Números enteros

Segundo Cuatrimestre 2025

Sistemas Digitales
DC - UBA

Introducción

Queremos representar una **magnitud** a través de un **sistema de representación**:

Queremos representar una **magnitud** a través de un **sistema de representación**:

- **Finito**

soporte fijo, cantidad de elementos acotados

Queremos representar una **magnitud** a través de un **sistema de representación**:

- **Finito**

soporte fijo, cantidad de elementos acotados

- **Composicional**

diversas magnitudes podrán representarse con un conjunto de elementos atómicos que deben ser fáciles de implementar y componer

Queremos representar una **magnitud** a través de un **sistema de representación**:

- **Finito**

soporte fijo, cantidad de elementos acotados

- **Composicional**

diversas magnitudes podrán representarse con un conjunto de elementos atómicos que deben ser fáciles de implementar y componer

- **Posicional**

la posición de cada dígito determina unívocamente en qué proporción modifica su valor a la magnitud total del número

Queremos representar una **magnitud** a través de un **sistema de representación**:

- **Finito**

soporte fijo, cantidad de elementos acotados

- **Composicional**

diversas magnitudes podrán representarse con un conjunto de elementos atómicos que deben ser fáciles de implementar y componer

- **Posicional**

la posición de cada dígito determina unívocamente en qué proporción modifica su valor a la magnitud total del número

El soporte formal lo encontraremos en las **bases de representación numérica**.

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

$$a = (b \times q_1 + r_1) \times b + r_0$$

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

$$a = (b \times q_1 + r_1) \times b + r_0$$

$$a = [(b \times q_2 + r_2) \times b + r_1] \times b + r_0$$

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

$$a = (b \times q_1 + r_1) \times b + r_0$$

$$a = [(b \times q_2 + r_2) \times b + r_1] \times b + r_0$$

Podemos continuar con la expansión hasta que $q_{N-1} < b$

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

$$a = (b \times q_1 + r_1) \times b + r_0$$

$$a = [(b \times q_2 + r_2) \times b + r_1] \times b + r_0$$

Podemos continuar con la expansión hasta que $q_{N-1} < b$

$$a = \{[(b \times q_{N-1} + r_{N-1}) \times b] + r_{N-2}\} \times b + r_0 \times 1$$

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

$$a = (b \times q_1 + r_1) \times b + r_0$$

$$a = [(b \times q_2 + r_2) \times b + r_1] \times b + r_0$$

Podemos continuar con la expansión hasta que $q_{N-1} < b$

$$a = \{[(b \times q_{N-1} + r_{N-1}) \times b] + r_{N-2}\} \times b + r_0 \times 1$$

Si distribuimos va a quedar:

Teorema:

Sean $a, b \in \mathbb{Z}$ con $b \neq 0$.

Existen $q, r \in \mathbb{Z}$ con $0 \leq r < |b|$

tales que $a = b \times q + r$

Además, q y r son únicos (*de a pares*).

¿Cómo lo usamos ?

$$a = b \times q_0 + r_0$$

$$a = (b \times q_1 + r_1) \times b + r_0$$

$$a = [(b \times q_2 + r_2) \times b + r_1] \times b + r_0$$

Podemos continuar con la expansión hasta que $q_{N-1} < b$

$$a = \{[(b \times q_{N-1} + r_{N-1}) \times b] + r_{N-2}\} \times b + r_0 \times 1$$

Si distribuimos va a quedar:

$$a = q_{N-1} \times b^N + r_{N-1} \times b^{N-1} + \dots + r_1 \times b + r_0 \times b^0$$

Representación posicional:

Podemos ver que el primer elemento de cada producto es el que va a aparecer en los dígitos de nuestra representación posicional, donde:

$$a = q_{N-1} \times b^N + r_{N-1} \times b^{N-1} + \dots + r_1 \times b + r_0 \times b^0$$

Se puede escribir como:

q_{N-1}	r_{N-1}	$\dots$	r_1	r_0
-----------	-----------	---------	-------	-------

O de forma correcta, incluyendo la base:

$$(q_{N-1}r_{N-1}\dots r_1r_0)_{(b)}$$

Ejemplo:

Veamos un ejemplo

Ejemplo:

$$42 \mid 3$$

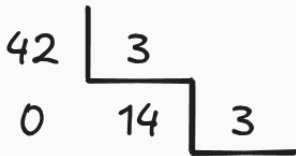


Ejemplo:

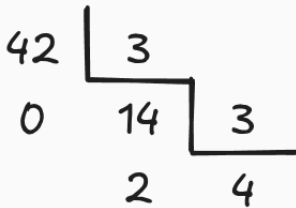
$$\begin{array}{r|l} 42 & 3 \\ \hline 0 & 14 \end{array}$$

Veamos un ejemplo

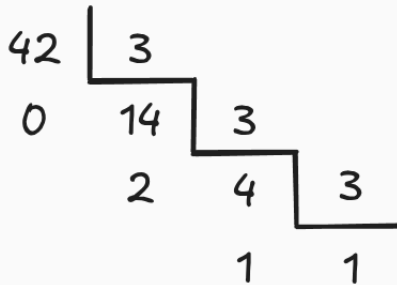
Ejemplo:



Ejemplo:

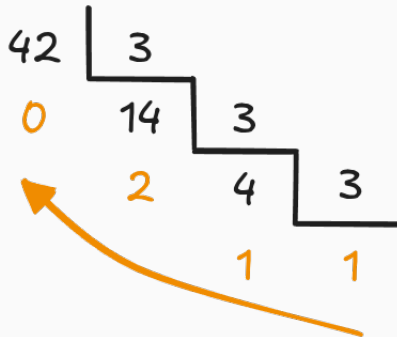


Ejemplo:



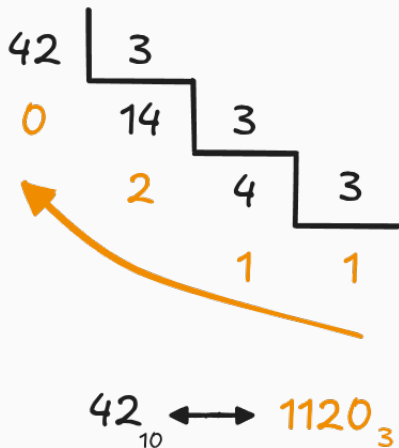
Veamos un ejemplo

Ejemplo:



Veamos un ejemplo

Ejemplo:



Representaciones numéricas

Sistemas de numeración

- Posicionales
 - Decimal: 2953253037_{10}

$$\begin{aligned} 2953253037_{10} = & 2 \cdot 10^9 + 9 \cdot 10^8 + 5 \cdot 10^7 + \\ & 3 \cdot 10^6 + 2 \cdot 10^5 + 5 \cdot 10^4 + \\ & 3 \cdot 10^3 + 3 \cdot 10^1 + 7 \cdot 10^0 \end{aligned}$$

Sistemas de numeración

- Posicionales
 - Decimal: 2953253037_{10}
 - Hexadecimal alias *hexa*: $2953253037_{10} = B00710AD_{16}$

$$B00710AD_{16} = B \cdot 16^7 + 7 \cdot 16^4 + 1 \cdot 16^3 + A \cdot 16^1 + D \cdot 16^0$$

Sistemas de numeración

- Posicionales

- Decimal: 2953253037_{10}

- Hexadecimal alias *hexa*: $2953253037_{10} = B00710AD_{16}$

- Sexagesimal: $2953253037_{10} = \text{𐎶 𐎠𐎺𐎠 𐎠𐎺𐎶 𐎠𐎺𐎶 𐎶 𐎠𐎺𐎠}_{60}$

$$\text{𐎶 𐎠𐎺𐎠 𐎠𐎺𐎶 𐎠𐎺𐎶 𐎶 𐎠𐎺𐎠}_{60} = \text{𐎶} \cdot 60^5 + \text{𐎠} \cdot 60^4 + \text{𐎺} \cdot 60^3 + \text{𐎠} \cdot 60^2 + \text{𐎶} \cdot 60^1 + \text{𐎠} \cdot 60^0$$

- Posicionales

- Decimal: 2953253037_{10}
- Hexadecimal alias *hexa*: $2953253037_{10} = \text{B00710AD}_{16}$
- Sexagesimal: $2953253037_{10} = \text{𐎠𐎼𐏁𐎧 𐏆𐏅𐏄 𐏇𐏂𐏈 𐏀𐏃𐏆𐏅 𐏁𐏍𐏗 𐏀𐏃𐏆𐏅}_{60}$
- Binario: $2953253037_{10} = 10110000000001110001000010101101_2$

$$1011000000001110001000010101101_2 =$$

$$1 \cdot 2^{31} + 0 \cdot 2^{30} + \dots +$$

$$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

... un tanto difícil de leer para más de 4 bits

Sistemas de numeración

- Posicionales
 - Decimal: 2953253037_{10}
 - Hexadecimal alias *hexa*: $2953253037_{10} = \text{B00710AD}_{16}$
 - Sexagesimal: $2953253037_{10} = \text{𐎶𐎵𐎴𐎳𐎲𐎱𐎰𐎯𐎮𐎭𐎬𐎫𐎪𐎩𐎨𐎧𐎦𐎥𐎤𐎣𐎢𐎡𐎠𐎟}_{60}$
 - Binario: $2953253037_{10} =$
 $10110000000001110001000010101101_2$

En general:

$$N_b = \sum_{i=-\infty}^{\infty} c_i \cdot b^i$$

Sistemas de numeración

- Posicionales

- Decimal: 2953253037_{10}

- Hexadecimal alias *hexa*: $2953253037_{10} = \text{B00710AD}_{16}$

- Sexagesimal: $2953253037_{10} = \text{𐎶 𐎵 𐎴 𐎳 𐎲 𐎱 𐎰 𐎯 𐎮 𐎭 𐎬 𐎫 𐎪 𐎩 𐎨 𐎧 𐎦 𐎥 𐎤 𐎣 𐎢 𐎡 𐎠 𐎟 𐎞 𐎝 𐎜 𐎛 𐎚 𐎙 𐎘 𐎗 𐎖 𐎕 𐎔 𐎓 𐎒 𐎑 𐎐 𐎏 𐎎 𐎍 𐎌 𐎋 𐎊 𐎉 𐎈 𐎇 𐎆 𐎅 𐎄 𐎃 𐎂 𐎁 𐎀 𐎠 𐎟 𐎞 𐎝 𐎜 𐎛 𐎚 𐎙 𐎘 𐎗 𐎖 𐎕 𐎔 𐎓 𐎒 𐎑 𐎐 𐎏 𐎎 𐎍 𐎌 𐎋 𐎊 𐎉 𐎈 𐎇 𐎆 𐎅 𐎄 𐎃 𐎂 𐎁 𐎀}_{60}$

- Binario: $2953253037_{10} =$
 $10110000000001110001000010101101_2$

En general:

$$N_b = \sum_{i=-\infty}^{\infty} c_i \cdot b^i$$

- No posicionales

- Romano: MMXIX = 2019_{10} 

Representación de números en Base 2

- El número 13548250_{10} se representa como $110011101011101011011010_2$

Representación de números en Base 2

- El número 13548250_{10} se representa como $110011101011101011011010_2$

$$N_2 = \sum_{i=0}^{m-1} c_i \cdot 2^i$$

Representación de números en Base 2

- El número 13548250_{10} se representa como $110011101011101011011010_2$

$$N_2 = \sum_{i=0}^{m-1} c_i \cdot 2^i$$

- El número $1,20_{10}$ se representa como $1.001100110011[0011]..._2$

$$\begin{aligned} 1,20_{10} = & 1 \cdot 2^0 + 0 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4} + 0 \cdot 2^{-5} + 0 \cdot 2^{-6} + 1 \cdot 2^{-7} + 1 \cdot 2^{-8} + 0 \cdot 2^{-9} + \\ & + 0 \cdot 2^{-10} + 1 \cdot 2^{-11} + 1 \cdot 2^{-12} + 0 \cdot 2^{-13} + 0 \cdot 2^{-14} + 1 \cdot 2^{-15} + 1 \cdot 2^{-16} + [0011] \dots_2 \end{aligned}$$

Representación de números en Base 2

- El número 13548250_{10} se representa como $110011101011101011011010_2$

$$N_2 = \sum_{i=0}^{m-1} c_i \cdot 2^i$$

- El número $1,20_{10}$ se representa como $1.001100110011[0011]..._2$

$$N_2 = \sum_{i=-n}^{m-n-1} c_i \cdot 2^i$$

$$\begin{aligned} 1,20_{10} = & 1 \cdot 2^0 + 0 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4} + 0 \cdot 2^{-5} + 0 \cdot 2^{-6} + 1 \cdot 2^{-7} + 1 \cdot 2^{-8} + 0 \cdot 2^{-9} + \\ & + 0 \cdot 2^{-10} + 1 \cdot 2^{-11} + 1 \cdot 2^{-12} + 0 \cdot 2^{-13} + 0 \cdot 2^{-14} + 1 \cdot 2^{-15} + 1 \cdot 2^{-16} + [0011] \dots_2 \end{aligned}$$

Representación de números en Base 2

- El número 13548250_{10} se representa como $110011101011101011011010_2$

$$N_2 = \sum_{i=0}^{m-1} c_i \cdot 2^i$$

- El número $1,20_{10}$ se representa como $1.001100110011[0011]..._2$

$$N_2 = \sum_{i=-n}^{m-n-1} c_i \cdot 2^i$$

$$1,20_{10} = 1 \cdot 2^0 + 0 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} + 1 \cdot 2^{-4} + 0 \cdot 2^{-5} + 0 \cdot 2^{-6} + 1 \cdot 2^{-7} + 1 \cdot 2^{-8} + 0 \cdot 2^{-9} + 0 \cdot 2^{-10} + 1 \cdot 2^{-11} + 1 \cdot 2^{-12} + 0 \cdot 2^{-13} + 0 \cdot 2^{-14} + 1 \cdot 2^{-15} + 1 \cdot 2^{-16} + [0011] \dots_2$$

- El número $1,3548250 \times 10^7$ se representa como

$$1.10011101011101011011010_2 \times 2^{23}$$

1 Byte = 8 bits

- Binario: 00000000_2 a 11111111_2
- Decimal: 0_{10} a 255_{10}
- Hexa: 00_{16} a FF_{16}
 - del 0 al 9 ¿y después? ¿113 es 1-13 u 11-3?

1 Byte = 8 bits

- Binario: 00000000_2 a 11111111_2
- Decimal: 0_{10} a 255_{10}
- Hexa: 00_{16} a FF_{16}
 - del 0 al 9 ¿y después? ¿113 es 1-13 u 11-3?

hexa	decimal	binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

1 Byte = 8 bits

- Binario: 00000000_2 a 11111111_2
- Decimal: 0_{10} a 255_{10}
- Hexa: 00_{16} a FF_{16}
 - del 0 al 9 ¿y después? ¿113 es 1-13 u 11-3?
 - Se utilizan los caracteres 0 a 9 y A a F

hexa	decimal	binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

1 Byte = 8 bits

- Binario: 00000000_2 a 11111111_2
- Decimal: 0_{10} a 255_{10}
- Hexa: 00_{16} a FF_{16}
 - del 0 al 9 ¿y después? ¿113 es 1-13 u 11-3?
 - Se utilizan los caracteres 0 a 9 y A a F
- En general, cuando programamos un entero en hexa se escribe de la siguiente manera:
 - $b00710ad_{16} \rightarrow 0xb00710ad$
 - o bien: $0xB00710AD$

hexa	decimal	binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

1 Byte = 8 bits

- Binario: 00000000_2 a 11111111_2
- Decimal: 0_{10} a 255_{10}
- Hexa: 00_{16} a FF_{16}
 - del 0 al 9 ¿y después? ¿113 es 1-13 u 11-3?
 - Se utilizan los caracteres 0 a 9 y A a F
- En general, cuando programamos un entero en hexa se escribe de la siguiente manera:
 - $b00710ad_{16} \rightarrow 0xb00710ad$
 - o bien: $0xB00710AD$

¿para qué me sirve?

hexa	decimal	binario
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Vector de bits:

$$X = (x_{w-1}, x_{w-2}, \dots, x_0)$$

$$x_i \in \{0, 1\}$$

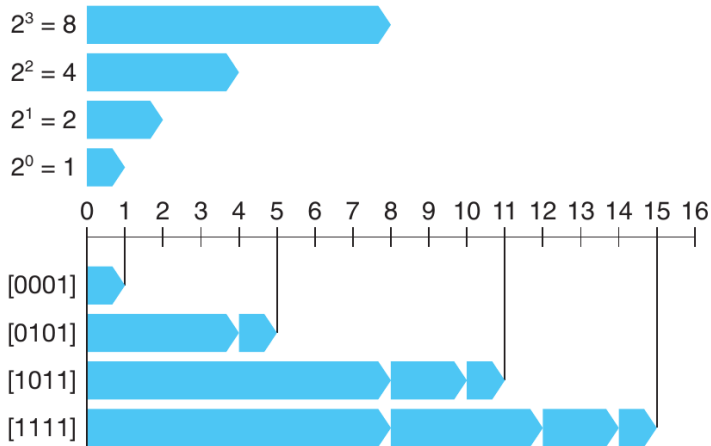
En representación sin signo:

$$B2U_w(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

Observaciones

- No hay bit de signo.
- Rango representable:

$$0 \leq B2U_w(X) \leq 2^w - 1$$



Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos
 - **Complemento a 2.**

Dado una palabra binaria de w bits, existen diversas formas de representar números:

- **Sin signo**
representa únicamente números positivos
- **Con signo**
 - **Signo + Magnitud**
se usa el primer dígito (bit) para indicar el signo de la magnitud
 - **Exceso m**
Represento n como $m + n$. Los valores a izquierda de m serán interpretados como negativos
 - **Complemento a 2.** ← Este es el importante de las representaciones con signo

Veamos ejemplos con rangos de representación para datos de 3 bits. El rango es:

v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
-------	-------	-------	-------	-------	-------	-------	-------

Los datos del rango son siempre iguales, lo que va a cambiar van a ser las magnitudes asociadas a cada elemento (cómo los interpretamos). Los datos son:

v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
000	001	010	011	100	101	110	111

Las magnitudes asociadas al rango serán:

Posición	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7
Dato	000	001	010	011	100	101	110	111
Sin signo	0	1	2	3	4	5	6	7
Signo+magnitud	0	1	2	3	-0	-1	-2	-3
Exceso $m(m=2)$	-2	-1	0	1	2	3	4	5

Noten que para signo+magnitud hay dos datos asociados al cero (el valor está desnormalizado).

Motivación

Es el método más utilizado para representar enteros con signo en computadoras. Su principal ventaja es que las operaciones aritméticas, como la resta, se simplifican enormemente.

- La resta de dos números ($A - B$) puede realizarse como la suma de A más el complemento a 2 de B .
- Existe una única representación para el cero.

$$\text{CA2}(b) = 2^n - b$$

Si sumamos a y $\text{CA2}(b)$:

$$a + \text{CA2}(b) = a + (2^n - b) = a - b + 2^n$$

Como trabajamos en aritmética de n bits, los cálculos se hacen módulo 2^n :

$$a - b + 2^n \equiv a - b \pmod{2^n}$$

Por lo tanto, tenemos que:

$$a - b \equiv a + \text{CA2}(b) \pmod{2^n}$$

Vector de bits:

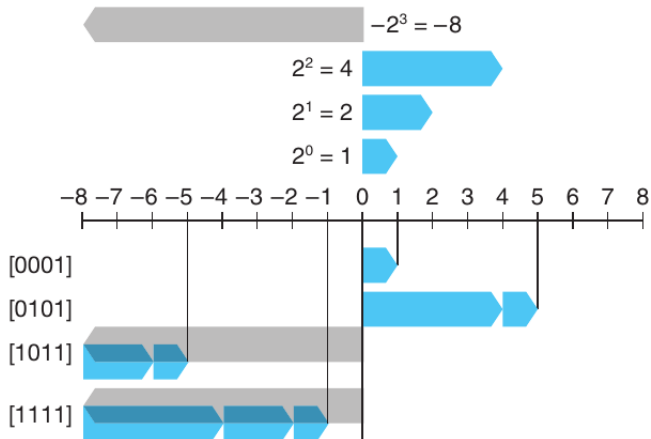
En complemento a 2:

$$X = (x_{w-1}, x_{w-2}, \dots, x_0) \quad B2T_w(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$
$$x_i \in \{0, 1\}$$

Observaciones

- El bit más significativo x_{w-1} actúa como *bit de signo*.
- Rango representable:

$$-2^{w-1} \leq B2T_w(X) \leq 2^{w-1} - 1$$



No signado

$$B2U_w(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

Complemento a 2

$$B2T_w(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

	Decimal	Hexa	Binario	
x	16162	3F 22	00111111	00100010
y	-16162	C0 DE	11000000	11011110

Bit de signo

En complemento a dos, el bit más significativo (MSB) indica el signo

- 0 para positivo
- 1 para negativo

Ejemplos $B2U_w(X)$ y $B2T_w(X)$

$x = 16162 : 00111111 \ 00100010$

$y = -16162 : 11000000 \ 11011110$

Peso (2^i)	16162		-16162	
1	0	0	0	0
2	1	2	1	2
4	0	0	1	4
8	0	0	1	8
16	0	0	1	16
32	1	32	0	0
64	0	0	1	64
128	0	0	1	128
256	1	256	0	0
512	1	512	0	0
1024	1	1024	0	0
2048	1	2048	0	0
4096	1	4096	0	0
8192	1	8192	0	0
16384	0	0	1	16384
-32768	0	0	1	-32768
Suma:	16162		-16162	

Límites: no signados

$$UMin = 0 \quad (000 \dots 0)$$

$$UMax = 2^w - 1 \quad (111 \dots 1)$$

Límites: complemento a dos

$$TMin = -2^{w-1} \quad (100 \dots 0)$$

$$TMax = 2^{w-1} - 1 \quad (011 \dots 1)$$

Valores para $w = 16$

	Decimal	Hexa		Binario	
UMax	65535	FF	FF	11111111	11111111
TMax	32767	7F	FF	01111111	11111111
TMin	-32768	80	00	10000000	00000000
-1	-1	FF	FF	11111111	11111111
0	0	00	00	00000000	00000000

	tamaño de palabra (w)			
	8	16	32	64
UMax	255	65535	4294967296	18 446 744 073 709 551 615
TMax	127	32767	2147483647	9 223 372 036 854 775 807
TMin	-128	-32768	-2147483648	-9 223 372 036 854 775 808

Observaciones

- $|TMin| = TMax + 1$
 - Rango asimétrico
- $UMax = 2 \cdot TMax + 1$

X	$B2U(X)$	$B2T(X)$
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

■ Equivalencia

- Mismo encoding para valores no negativos

■ Unicidad

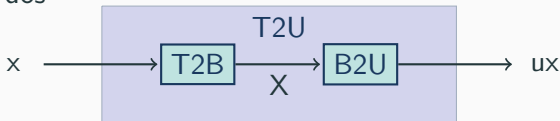
- Cada patrón representa un único entero
- A cada entero representable le corresponde un único patrón

■ $\Rightarrow$ se pueden invertir los mapeos

- $U2B(X) = B2U^{-1}(X)$
 - Patrón de bits para un entero sin signo
- $T2B(X) = B2T^{-1}(X)$
 - Patrón de bits para un entero en complemento a

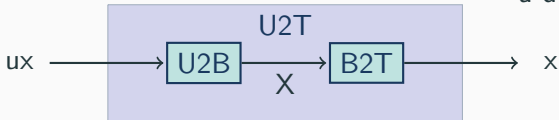
complemento
a dos

no signado



no signado

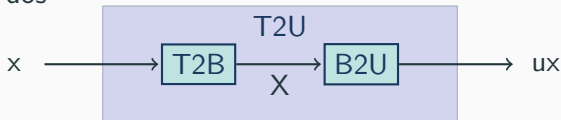
complemento
a dos



No cambia el patrón de bits: **mantener los bits y reinterpretar**

complemento
a dos

no signado



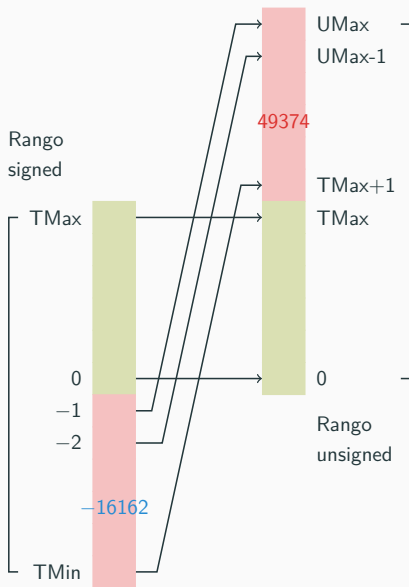
$$\begin{array}{r}
 \begin{array}{c} w-1 \qquad \qquad \qquad 0 \\ ux \end{array}
 \begin{array}{|c|c|c|c|c|c|c|}
 \hline
 + & + & + & \dots & + & + & + \\
 \hline
 \end{array} \\
 - \quad x \quad
 \begin{array}{|c|c|c|c|c|c|c|}
 \hline
 - & + & + & \dots & + & + & + \\
 \hline
 \end{array} \\
 \hline
 +2^{w-1} - -2^{w-1} = 2 \cdot 2^{w-1} = 2^w
 \end{array}$$

$$ux = \begin{cases} x & x \geq 0 \\ x + 2^w & x < 0 \end{cases}$$

Un peso negativo grande se convierte en un peso positivo grande

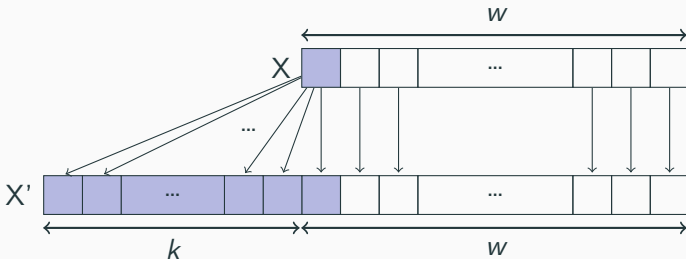
Relación entre signado y no signado

Peso (2^i)	-16162		49374	
1	0	0	0	0
2	1	2	1	2
4	1	4	1	4
8	1	8	1	8
16	1	16	1	16
32	0	0	0	0
64	1	64	1	64
128	1	128	1	128
256	0	0	0	0
512	0	0	0	0
1024	0	0	0	0
2048	0	0	0	0
4096	0	0	0	0
8192	0	0	0	0
16384	1	16384	1	16384
± 32768	1	-32768	1	32768
Suma:	-16162		49374	



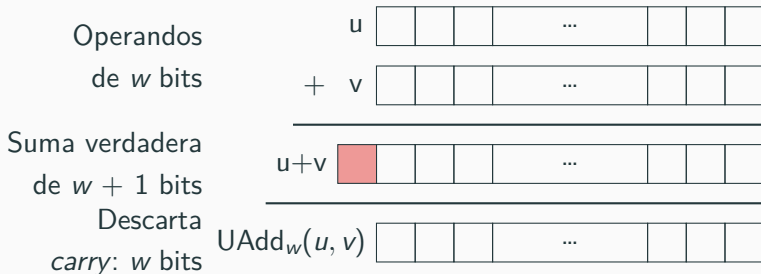
- Queremos:
 - Dado un entero *signado* de w -bits
 - Convertirlo a un entero de $w + k$ -bits con el mismo valor
- Regla:
 - Propagar el bit de signo a los nuevos k bits (MSB)

$$X' = \underbrace{x_{w-1}, \dots, x_{w-1}}_{k \text{ copias del MSB}}, \underbrace{x_{w-1}, x_{w-2}, \dots, x_1, x_0}_{\text{entero original}}$$



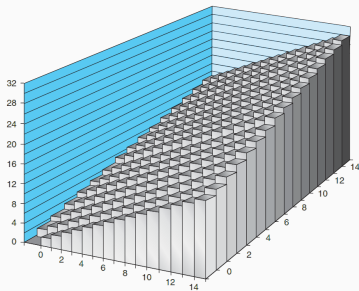
	Decimal	Hex	Binario
x	16162	3F 22	00111111 00100010
ix	16162	00 00 3F 22	00000000 00000000 00111111 00100010
y	-16162	C0 DE	11000000 11011110
iy	-16162	FF FF C0 DE	11111111 11111111 11000000 11011110

Se aumenta la cantidad de bits en la representación: expansión ó *up casting*.

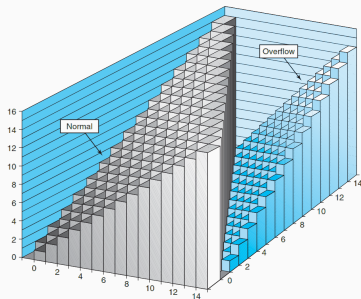


$$\text{UAdd}_w(u, v) = u + v \text{ mód } 2^w$$

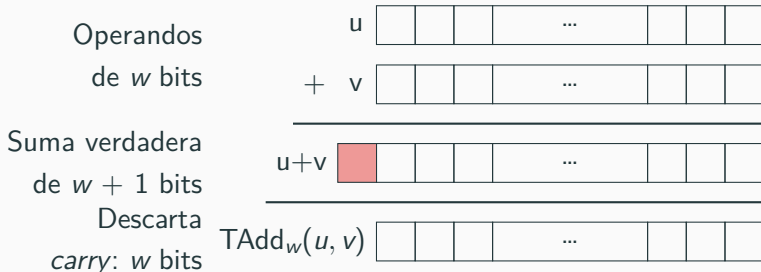
Suma de enteros no signados



Suma teórica



Suma real

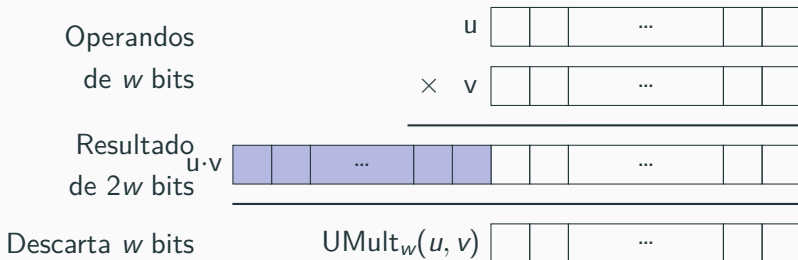


- **A nivel binario: comportamiento idéntico a la suma unsigned**

Ejemplo con $w = 4 \implies$

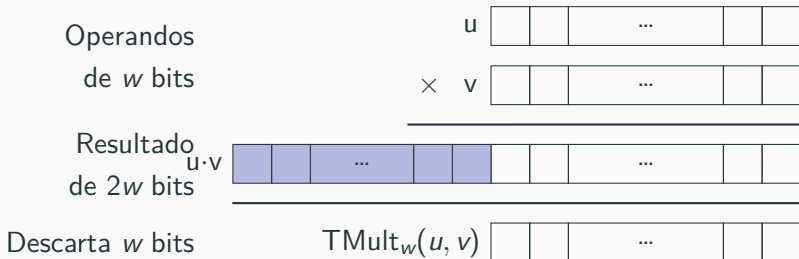
- Requiere $w + 1$ bits
- Descarta el MSB
- El resultado es en complemento a dos
- *wraps around*
 - Si la suma $\geq 2^{w-1}$
 - Se vuelve negativa
 - No más de una vez
 - Si la suma $< -2^{w-1}$
 - Se vuelve positiva
 - No más de una vez

u		v		suma		TAdd ₄ (u,v)		
bin	dec	bin	dec	bin	dec	msb	4-bit	dec
0111	7	0111	7	01110	14	0	1110	-2
0111	7	0110	6	01101	13	0	1101	-3
0111	7	0101	5	01100	12	0	1100	-4
0110	6	0100	4	01010	10	0	1010	-6
0011	3	0110	6	01001	9	0	1001	-7
0110	6	0010	2	01000	8	0	1000	-8
0100	4	0011	3	00111	7	0	0111	7
0000	0	0110	6	00110	6	0	0110	6
0101	5	0000	0	00101	5	0	0101	5
1101	-3	0110	6	00011	3	0	0011	3
1111	-1	0010	2	00001	1	0	0001	1
1001	-7	0111	7	00000	0	0	0000	0
0011	3	1100	-4	11111	-1	1	1111	-1
1000	-8	0101	5	11101	-3	1	1101	-3
0100	4	1000	-8	11100	-4	1	1100	-4
1001	-7	0010	2	11011	-5	1	1011	-5
1000	-8	0001	1	11001	-7	1	1001	-7
1010	-6	1110	-2	11000	-8	1	1000	-8
1000	-8	1111	-1	10111	-9	1	0111	7
1010	-6	1100	-4	10110	-10	1	0110	6
1001	-7	1100	-4	10101	-11	1	0101	5
1000	-8	1100	-4	10100	-12	1	0100	4
1000	-8	1001	-7	10001	-15	1	0001	1
1000	-8	1000	-8	10000	-16	1	0000	0



- **Multiplicación estándar**
 - ignora los w bits superiores
- **Implementa aritmética modular**
 - como ocurre con la suma:

$$UMult_w(u, v) = u \cdot v \mod 2^w$$



■ Multiplicación estándar

- ignora los w bits superiores
- los bits dentro de los w superiores pueden diferir
- los bits inferiores son iguales a $\text{UMult}_w(u, v)$

- **Representación numérica**

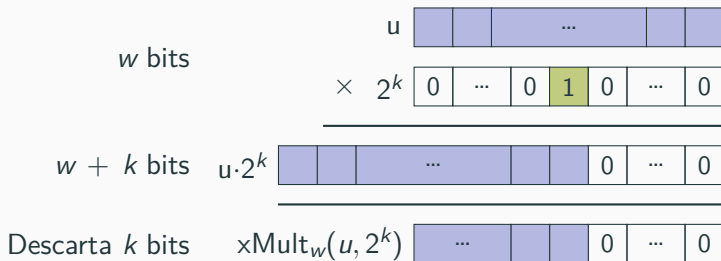
$$N_2 = \sum_{i=0}^{w-1} b_i \cdot 2^i$$

- **Multiplicar por 2^k**

- requiere k bits más

$$\begin{aligned} N_2 \cdot 2^k &= \left(\sum_{i=0}^{w-1} b_i \cdot 2^i \right) \cdot 2^k \\ &= \sum_{i=0}^{w-1} b_i \cdot 2^{i+k} \\ &= \sum_{i=k}^{w+k-1} b_i \cdot 2^i + \sum_{j=0}^{k-1} 0 \cdot 2^j \end{aligned}$$

- ignora los k bits superiores



Operación

- $u \cdot 2^k$ se puede hacer con desplazamientos: $u \ll k$
- Tanto para signados como para no signados

Operación

- $\lfloor u/2^k \rfloor$ se puede hacer con desplazamientos: $u \gg k$
- Tanto para signados como para no signados

Ejemplos

	Teórica	Real	Hex	Binario
x	16162	16162	3F 22	00111111 00100010
x >> 1	8081.0	8081	1F 91	00011111 10010001
x >> 2	4040.50	4040	0F C8	00001111 11001000
x >> 8	63.133	63	00 3F	00000000 00111111
y	-16162	-16162	C0 DE	11000000 11011110
y >> 1	-8081.0	-8081	E0 6F	11100000 01101111
y >> 2	-4040.50	-4041	F0 37	11110000 00110111
y >> 8	-63.133	-64	FF C0	11111111 11000000

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	?		
-2			
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011		
-2			
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	?	
-2			
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	
-2			
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	?
-2			
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2			
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	?		
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010		
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	?	
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	?
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8			

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8	?		

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8	OVERFLOW		

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8	OVERFLOW	?	

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8	OVERFLOW	1000	

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8	OVERFLOW	1000	?

En base 2, datos de 4 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0011	0011	OVERFLOW
-2	1010	1110	1101
-8	OVERFLOW	1000	0111

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	?		
-2			
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011		
-2			
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	?	
-2			
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	
-2			
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	?
-2			
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2			
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	?		
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010		
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	?	
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	?
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8			

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	?		

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000		

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000	?	

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000	1111 1000	

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000	1111 1000	?

En base 2, datos de 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000	1111 1000	0000 0111

Similitudes entre 4 y 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000	1111 1000	0000 0111

Similitudes entre 4 y 8 bits

	Signo + Magnitud	Complemento a 2	Exceso a 15
3	0000 0011	0000 0011	0001 0010
-2	1000 0010	1111 1110	0000 1101
-8	1000 1000	1111 1000	0000 0111

Extendiendo la cantidad de bits de precisión:

- Signo + Magnitud: Se extiende con 0's, pero el bit más significativo se mantiene indicando el signo.
- Complemento a 2: Se extiende con el valor del bit más significativo.
- Exceso a m: Se extiende siempre con 0's.

Transformando los datos

Vamos a presentar algunas operaciones aritméticas y lógicas en base 2, en particular **o lógico, y lógico, xor lógico, negación lógica, desplazamientos, suma, resta y multiplicación**. Todas las operaciones van a estar asociadas a un algoritmo de resolución, del mismo modo que utilizamos uno para realizar cuentas en base 10 en la vida cotidiana.

Es importante comprender que hay atributos que nos interesa observar tanto de **los datos como de las operaciones**. Por ejemplo:

- **De los datos numéricos** si son negativos o pares
- **De las operaciones** si los resultados se mantienen dentro del rango de representación

Veamos como observar el dato nos permite determinar propiedades del valor representado. Por ejemplo, en complemento a 2 de un dato de 4 bits:

Posición	v_3	v_2	v_1	v_0
Interpretación	<i>signo</i>	x	x	<i>paridad</i>
Negativo	1	x	x	x
Par	x	x	x	0

Las operaciones lógicas que veremos pueden involucrar a uno o dos operandos, **se aplican sobre el dato almacenado, o sea los bits del valor representado.**

Las presentamos rápidamente y una por una.

El **o lógico** o disyunción se aplica bit a bit. La operación $a \vee b = c$ se puede describir atómicamente (por cada elemento indivisible) como $c_i = a_i \vee b_i$. Veamos un ejemplo de 4 bits, noten que la aplicación no depende del tipo de dato, lo trata indistintamente:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
b	0	0	1	1
$c = a \vee b$	1	0	1	1

Podemos notar que las operaciones lógicas se aplican **bit a bit**:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
	+	+	+	+
b	0	0	1	1
	↓	↓	↓	↓
$c = a \vee b$	1	0	1	1

Es posible encontrar al signo $+$ para representar la disyunción, porque equivale a una suma sin acarreo en bits. Lo mismo sucede entre la conjunción y el signo $*$.

El **y lógico** se aplica bit a bit. La operación $a \wedge b = c$ se puede describir atómicamente (por cada elemento indivisible) como $c_i = a_i \wedge b_i$. Veamos un ejemplo de 4 bits:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
b	0	0	1	1
$c = a \wedge b$	0	0	1	0

El **xor lógico** se aplica bit a bit. La operación $a \vee b = c$ se puede describir atómicamente (por cada elemento indivisible) como $c_i = (a_i \wedge \neg b_i) \vee (\neg a_i \wedge b_i)$, exactamente uno de los bits vale 1. Veamos un ejemplo de 4 bits:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
b	0	0	1	1
$c = a \vee b$	1	0	0	1

La **negación lógica** se aplica bit a bit. La operación $\neg a = c$ se puede describir atómicamente (por cada elemento indivisible) como $c_i = \neg a_i$, dando vuelta los valores de cada elemnto. Veamos un ejemplo de 4 bits:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
$c = \neg a$	0	1	0	1

El **desplazamiento a izquierda** se aplica desplazando los bits del dato tantas posiciones como se indiquen a izquierda. La operación $a \ll n = c$ para un dato de k bits se puede describir atómicamente como $c_i = a_{i-n}$ si $n \leq i \leq k-1$ y 0 en caso contrario. Veamos un ejemplo de 4 bits:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
$c = a \ll 2$	1	0	0	0

El **desplazamiento lógico a derecha** se aplica desplazando los bits del dato tantas posiciones como se indiquen a derecha. La operación $a \gg_r n = c$ para un dato de k bits se puede describir atómicamente como $c_i = a_{i+n}$ si $0 \leq i \leq k - 1 - n$ y 0 en caso contrario. Veamos un ejemplo de 4 bits:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
$c = a \gg_r 2$	0	0	1	0

El **desplazamiento aritmético a derecha** se aplica desplazando los bits del dato tantas posiciones como se indiquen a derecha, pero copiando el valor del bit más significativo de origen en los valores vacantes del resultado. La operación $a \gg_a n = c$ para un dato de k bits se puede describir atómicamente como $c_i = a_{i+n}$ si $0 \leq i \leq k-1-n$ y a_{k-1} en caso contrario. Veamos un ejemplo de 4 bits:

Posición	v_3	v_2	v_1	v_0
a	1	0	1	0
$c = a \gg_a 2$	1	1	1	0

¿Por qué existe una diferencia entre el desplazamiento a derecha lógico y aritmético? La aplicación de una operación lógica puede tener muchos motivos, pero en particular el desplazamiento a derecha o izquierda en n posiciones **tiene el efecto de multiplicar o dividir por 2^n el valor representado**, siempre que se trate de un entero sin signo o en complemento a dos.

Si a es un valor representado en complemento a dos, ¿cuándo valen las siguientes propiedades?

$$a \gg_l n = a/2^n$$

$$a \gg_a n = a/2^n$$

Ejercicio para la casa.

Técnicas de uso corriente



Pasar el número 28 y a binario.

Si hacemos divisiones
sucesivas:

¿Cómo pasamos rápido de decimal a binario?



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Pasar el número 28 y a binario.

Si hacemos divisiones
sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número
11100.

Otra aproximación.

Busco $2^x \geq 28$



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 *$$

$$2^2 + x_4 * 2^1 + x_5 * 2^0$$



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 *$$

$$2^2 + x_4 * 2^1 + x_5 * 2^0$$

--	--	--	--	--



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 * 2^2 + x_4 * 2^1 + x_5 * 2^0$$

$$2^2 + x_4 * 2^1 + x_5 * 2^0$$

1				
---	--	--	--	--



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya
que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 * 2^2 + x_4 * 2^1 + x_5 * 2^0$$

1	1			
---	---	--	--	--



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 *$$

$$2^2 + x_4 * 2^1 + x_5 * 2^0$$

1	1	1		
---	---	---	--	--



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya

que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 * 2^2 + x_4 * 2^1 + x_5 * 2^0$$

$$2^2 + x_4 * 2^1 + x_5 * 2^0$$

1	1	1	0	
---	---	---	---	--



Pasar el número 28 y a binario.

Si hacemos divisiones

sucesivas:

$$28 / 2 = 14 \quad \text{Resto} = 0$$

$$14 / 2 = 7 \quad \text{Resto} = 0$$

$$7 / 2 = 3 \quad \text{Resto} = 1$$

$$3 / 2 = 1 \quad \text{Resto} = 1$$

Quedando así el número

11100.

Otra aproximación.

Busco $2^x \geq 28 \rightarrow 5$, ya
que $2^5 = 32$. Necesitaré 5
dígitos

Sabemos:

$$x_1 * 2^4 + x_2 * 2^3 + x_3 * 2^2 + x_4 * 2^1 + x_5 * 2^0$$

1	1	1	0	0
---	---	---	---	---

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16
- Luego es potencia de 2 directo, 2^4

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16
- Luego es potencia de 2 directo, 2^4
- o sea puedo separarlos de a 4:

1010	1100
A	C

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16
- Luego es potencia de 2 directo, 2^4
- o sea puedo separarlos de a 4:

1010	1100
A	C

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16
- Luego es potencia de 2 directo, 2^4
- o sea puedo separarlos de a 4:

1010	1100
A	C

¿Y para pasar de hexadecimal a binario?

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16
- Luego es potencia de 2 directo, 2^4
- o sea puedo separarlos de a 4:

1010	1100
A	C

¿Y para pasar de hexadecimal a binario?

¿Y para pasar a complemento a 2?

Pasar el número 10101100

- ¿Cuánto elementos puedo representar con un dígito hexadecimal?
- 16
- Luego es potencia de 2 directo, 2^4
- o sea puedo separarlos de a 4:

1010	1100
A	C

¿Y para pasar de hexadecimal a binario?

¿Y para pasar a complemento a 2? Hay que hacer la guía...

El truco para detectar un overflow es observar que si el bit de signo es igual en ambos operandos (ambos positivos o negativos) el resultado de la suma debería preservar el signo (suma de positivos produce un positivo, suma de negativos produce un negativo).

$$\text{overflow} \iff ((a_{n-1} = b_{n-1}) \wedge (a_{n-1} \neq c_{n-1}))$$

Algunos ejemplos para pensar **en C2** (complemento a 2)...

Algunos ejemplos para pensar **en C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \\ \hline 10010 \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \end{array}$$

OK!

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \\ \hline 11000 \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

$$-5 - 4 = (-5) + (-4) = -9$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1011 \\ + 1100 \\ \hline 10111 \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1011 \\ + 1100 \\ \hline 10111 \\ (C \leftarrow \overline{C}) \\ \text{OVERFLOW!} \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1011 \\ + 1100 \\ \hline 10111 \\ (C \leftarrow \overline{C}) \\ \text{OVERFLOW!} \end{array}$$

$$5 + 4 = 9$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1011 \\ + 1100 \\ \hline 10111 \\ (C \leftarrow \overline{C}) \\ \text{OVERFLOW!} \end{array}$$

$$5 + 4 = 9$$

$$\begin{array}{r} 0101 \\ + 0100 \\ \hline 01001 \end{array}$$

Algunos ejemplos para pensar en **C2** (complemento a 2)...

$$5 - 3 = 5 + (-3) = 2$$

$$\begin{array}{r} 0101 \\ + 1101 \quad (C \leftarrow C) \\ \hline 10010 \\ \text{OK!} \end{array}$$

$$-5 - 3 = (-5) + (-3) = (-8)$$

$$\begin{array}{r} 1011 \\ + 1101 \quad (C \leftarrow C) \\ \hline 11000 \\ \text{OK!} \end{array}$$

$$-5 - 4 = (-5) + (-4) = -9$$

$$\begin{array}{r} 1011 \\ + 1100 \\ \hline 10111 \\ (C \leftarrow \bar{C}) \\ \text{OVERFLOW!} \end{array}$$

$$5 + 4 = 9$$

$$\begin{array}{r} 0101 \\ + 0100 \quad (\bar{C} \leftarrow C) \\ \hline 01001 \\ \text{OVERFLOW!} \end{array}$$

Noten que el **overflow** y el **carry** son propiedades de una operación, a diferencia de la paridad o el signo, que son propiedades de un dato aislado. Para determinar si se cumple una propiedad de una operación, puede ser necesario observar tanto los operandos como el resultado (caso del overflow).

Cierre

Resumen de la clase práctica del día de la fecha.

- Motivación, características y necesidad de contar con un sistema de representación de la información sobre un soporte dado. **Representación numérica en base 2.**
- Formas de representar tipos numéricos, **naturales (sin signo), enteros (signo+magnitud, exceso m, complemento a 2).**
- Operaciones lógico-aritméticas utilizadas para transformar la información (**o lógico, y lógico, xor lógico, negación lógica, desplazamientos, suma, resta y multiplicación**) y sus propiedades asociadas (**carry y overflow**).

- ¡Ya se puede hacer la práctica 1 completa!.
- Circuitos Combinatorios.
- Circuitos Secuenciales.

Pasamos a las preguntas.